

Module 5: Lesson 4

Bidirectional LSTMs and BTC prices



Outline

- ▶ The prediction problem at hand
- ▶ Bi-LSTM network for BTC returns

The prediction problem

Here, we repeat the same analysis we performed in the notebook of Lesson 2 of this module. In this case, however, we will deal with bidirectional LSTM (Bi-LSTM) instead of unidirectional ones. Our interest, one more time, lies in predicting short-term future asset returns:

- ▶ Which asset? **Bitcoin (BTC)**
- ▶ Which returns? **25-hour ahead returns**
- ▶ Which inputs? Past cumulative returns for **1-hour, 10-hour, and 50-hour**
- ▶ Which period? **Jan-2021 to Sept-2022**

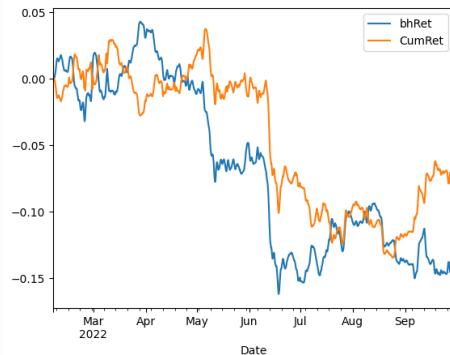
	date	Ret	Ret_10	Ret_50	Ret25
52634	2021-01-03 01:00:00	0.003812	0.000300	0.001236	0.000294
52635	2021-01-03 02:00:00	0.004669	0.000103	0.001244	0.000270
52636	2021-01-03 03:00:00	-0.004577	-0.000065	0.001066	0.000233
52637	2021-01-03 04:00:00	0.019479	0.000120	0.001334	-0.000088
52638	2021-01-03 05:00:00	0.014644	0.000235	0.001445	-0.000118

Bi-LSTM model to predict BTC returns

In the notebook accompanying this Lesson, we will guide you through the development of a Bi-LSTM model with bi. Here you have the summary of the model, as well as the out-of-sample performance of a backtest for the strategy using model predictions:

Layer (type)	Output Shape	Param #
bidirectional (BidirectionalL)	(None, 30, 100)	21600
bidirectional_1 (BidirectionalL)	(None, 30, 100)	60400
dropout (Dropout)	(None, 30, 100)	0
bidirectional_2 (BidirectionalL)	(None, 30, 100)	60400
dropout_1 (Dropout)	(None, 30, 100)	0
bidirectional_3 (BidirectionalL)	(None, 30, 100)	60400
dropout_2 (Dropout)	(None, 30, 100)	0
bidirectional_4 (BidirectionalL)	(None, 100)	60400
dropout_3 (Dropout)	(None, 100)	0
dense (Dense)	(None, 20)	2020
dropout_4 (Dropout)	(None, 20)	0
dense_1 (Dense)	(None, 10)	210
dropout_5 (Dropout)	(None, 10)	0
dense_2 (Dense)	(None, 1)	11

=====
Total params: 265,441
Trainable params: 265,441
Non-trainable params: 0



Summary of Lesson 4

In Lesson 4, we have:

- ▶ Developed our first bidirectional LSTM Neural Network model
- ▶ Compared the complexity of bidirectional models with unidirectional ones
- ▶ Evaluated the predictive power of bidirectional LSTM and backtest the strategy in test sample.

TO DO NEXT: Go to the accompanying Python Notebook to implement your first bidirectional LSTM!

In the next module, we will introduce Autoencoders, an example of unsupervised neural network.

DEEP LEARNING FOR FINANCE

MODULE 5 | LESSON 2

LSTMs FOR PREDICTING BTC PRICES

Reading Time	55 minutes
Prior Knowledge	LSTM, Backtesting, Market timing
Keywords	LSTM, BTC, EDA

1. Introduction

In this lesson, we will develop our first LSTM neural network using high-frequency data on Bitcoin prices. This way, we will also experiment with relatively high frequency data (we will have hourly observations) and how to deal with some of the issues this type of data imposes. Moreover, we will work with a reduced dataset during the lesson to minimize computational time, but you will have the complete dataset available to play around with and see how different models perform for other chunks of data.

At this point, it is likely that all of you are quite familiar with Bitcoin (BTC) prices and how they are expressed. Since BTC is a currency after all, its prices are most commonly expressed in dollar terms per unit of BTC currency. This means that when we say BTC is at 20,000, we mean 1 unit of BTC could be exchanged for 20,000 USD. This is somewhat obvious but sometimes important to keep in mind, since the different risks affecting the price of BTC can stem from different sources that affect the 'pair' BTC-USD.

The specific data we are going to work with in this lesson comes from Kraken.com (<https://www.kraken.com>). This is essentially a crypto-asset exchange that also makes available transaction data on different "coins" and NFTs. **Please, by no means should you interpret this as a recommendation to invest in BTC, crypto, or NFTs.** We are just going to leverage this data to have a little fun while investigating whether LSTM do a decent enough job in predicting high-frequency BTC prices.

So, let's get to it!

2. Data

As usual, let's start by importing our data and cleaning if necessary:

First, the libraries that we will need,

```
In [1]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import tensorflow as tf
from keras.layers import LSTM, Dense, Dropout
from sklearn.preprocessing import MinMaxScaler
```

```
2025-05-14 12:53:17.428018: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-05-14 12:53:17.498641: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
2025-05-14 12:53:19.344432: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
```

Then, let's set up our interface to import the data:

```
In [2]: drivepath = "Kraken_BTCUSD_60_3Q2022.csv"
```

```
In [3]: df = pd.read_csv(drivepath)

# Given the format of the data, we need to also
df["unix"] = pd.to_numeric(df["unix"])
df["date"] = pd.to_datetime(df["unix"], unit="s")
```

How does our data look?

```
In [4]: df
```

Out[4]:

	unix	open	high	low	close	volume	trades
0	1381093200	122.00000	122.00000	122.00	122.00	0.100000	1
1	1381176000	123.61000	123.61000	123.61	123.61	0.100000	1
2	1381197600	123.91000	123.91000	123.90	123.90	1.991600	2
3	1381208400	124.19000	124.19000	124.18	124.18	2.000000	2
4	1381309200	124.01687	124.01687	123.84	123.84	2.823000	3
...	...	...	...	...	...	...	...
67886	1664564400	19673.10000	19762.90000	19482.80	19497.20	290.023381	1947
67887	1664568000	19491.00000	19535.70000	19351.20	19420.40	759.074836	1648
67888	1664571600	19423.60000	19423.60000	19281.00	19360.40	564.894247	1245
67889	1664575200	19354.20000	19399.90000	19232.60	19387.10	90.626135	1978
67890	1664578800	19374.80000	19476.40000	19364.90	19425.20	57.986112	2466

67891 rows × 8 columns



As you can see, there is a lot of information in our hourly data on BTC prices. We have data on open, high, low, and close prices for each hour between October 2013 and September 2022. Since this data is somewhat special compared to our 'usual' stock and index prices, it deserves a little more exploration before jumping to the prediction problem using LSTMs.

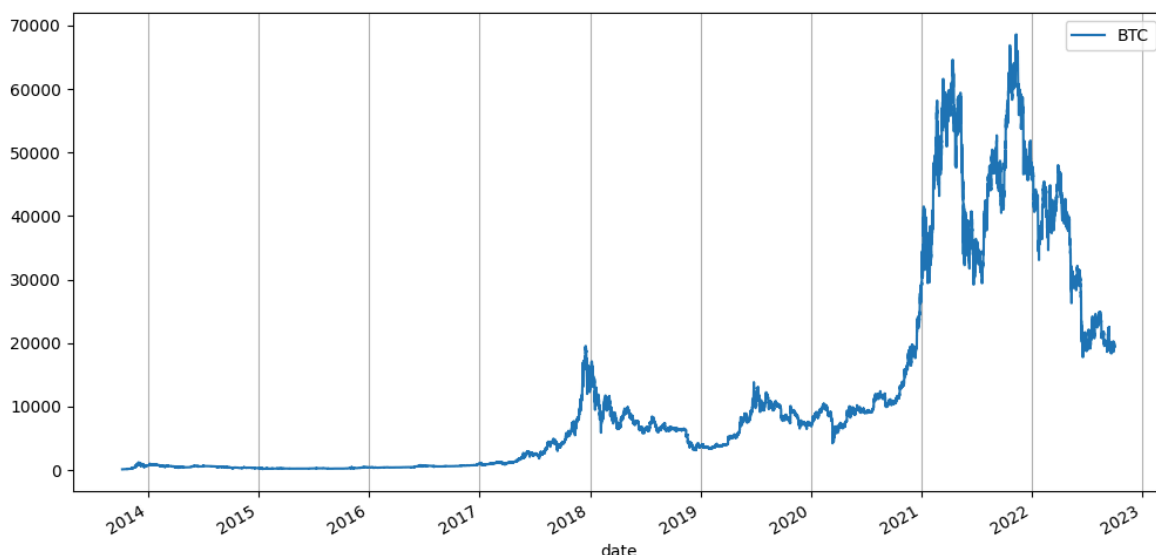
3. Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is a useful process in these cases in which we are not too familiar with the data we are dealing with. It basically consists of exploring the data,

specially via visualization, which simplifies the task of learning how the data behaves. This will be relevant later on to decide the kind of scaling we need, to give more or less importance to the presence of outliers, etc.

We will start by simply looking at the hourly closing prices throughout the whole period:

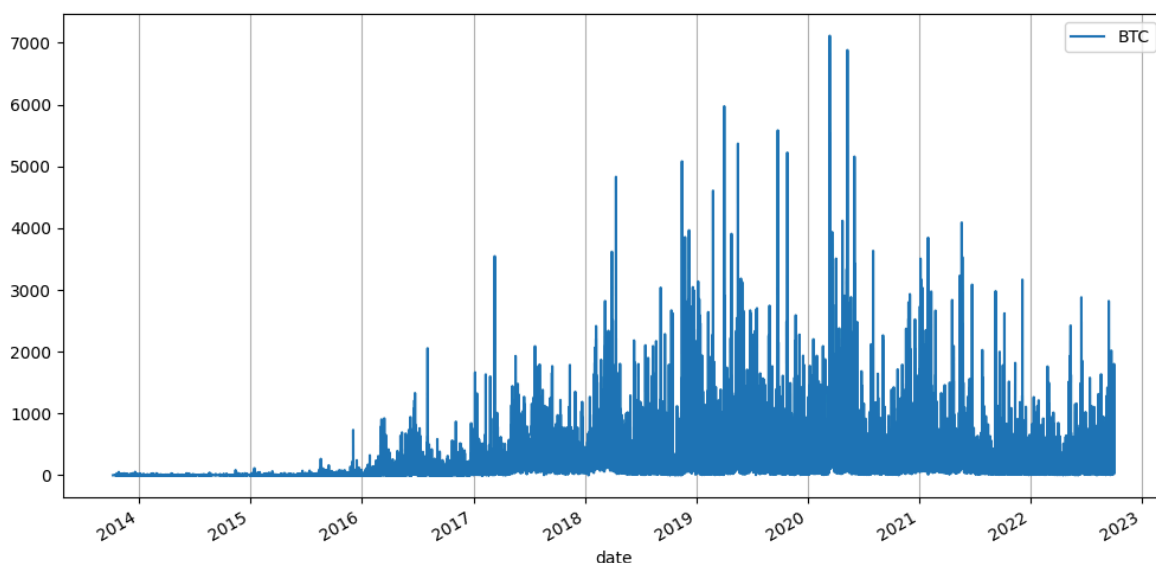
```
In [5]: fig = plt.figure(figsize=(12, 6))
ax = plt.gca()
df.plot(x="date", y="close", label="BTC", ax=ax).grid(axis="x")
```



From the previous graph, we observe a clear upward trend in the last few years that, nonetheless, has stopped in more recent years. BTC prices are now in the neighborhood of 20,000 USD, far from its maximum of nearly 70,000 USD by the final months of 2022.

Has this increase in prices been accompanied by similar increases in volume?

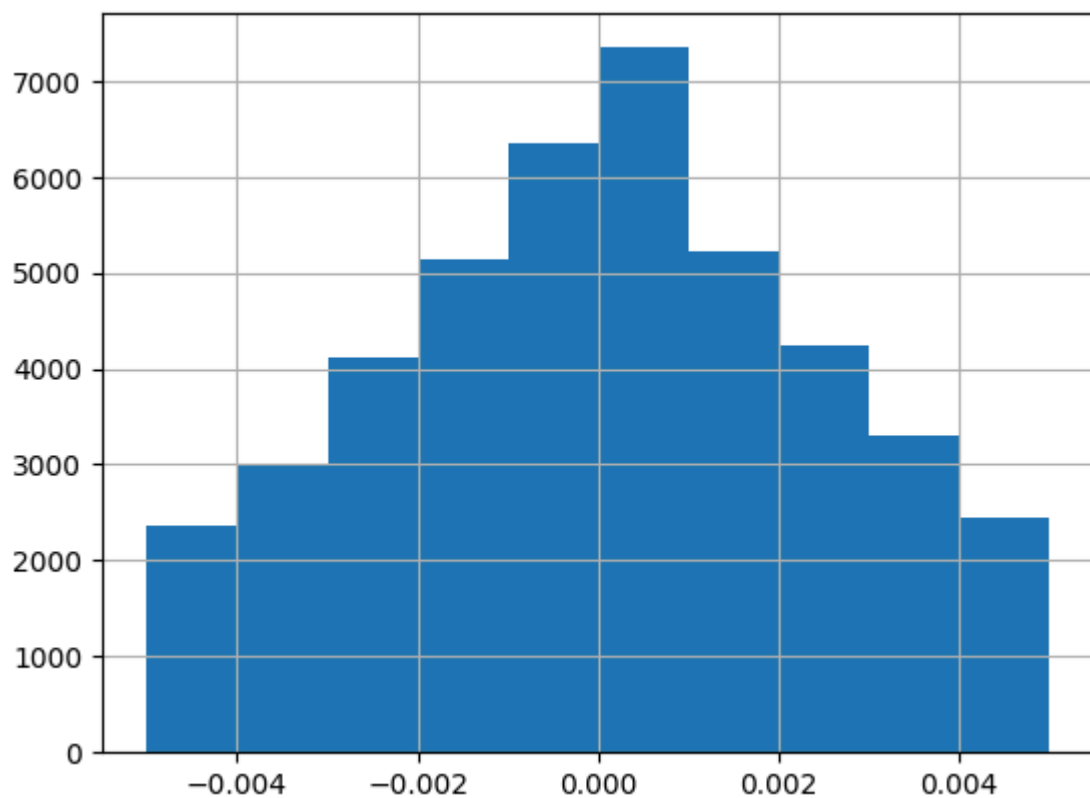
```
In [6]: fig = plt.figure(figsize=(12, 6))
ax = plt.gca()
df.plot(x="date", y="volume", label="BTC", ax=ax).grid(axis="x")
```



What about the distribution of hourly returns?

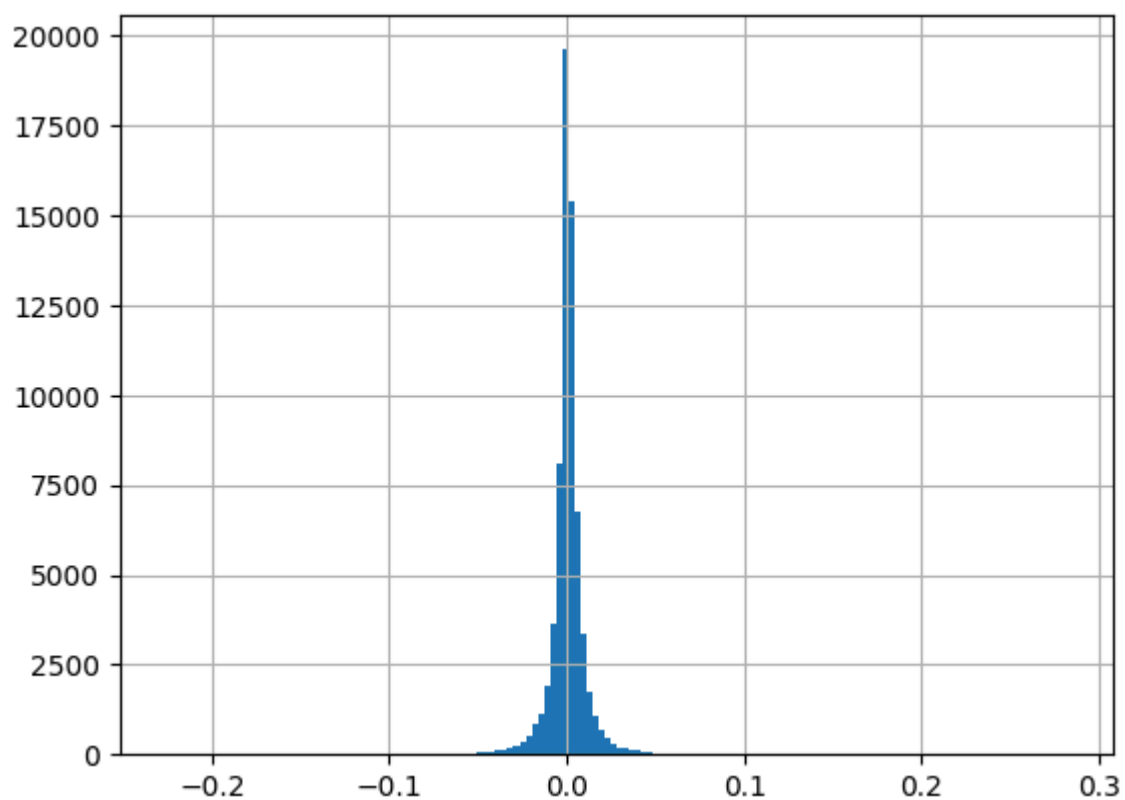
```
In [7]: df["Ret"] = df["close"].pct_change()  
df["Ret"].hist(range=[-0.005, 0.005])
```

Out[7]: <Axes: >



```
In [8]: df["Ret"].hist(bins=150)
```

Out[8]: <Axes: >



Well, indeed, hourly returns seem pretty centered around zero, resembling a normally distributed variable. Is this the case though?

```
In [9]: df["Ret"].describe()
```

```
Out[9]: count    67890.000000
      mean         0.000140
      std         0.011395
      min        -0.226804
      25%        -0.003045
      50%         0.000016
      75%         0.003381
      max         0.282090
      Name: Ret, dtype: float64
```

Well, so far, we have looked at the raw data (i.e., in hourly frequency). But maybe it is also interesting to see how the data behaves when we aggregate it at other frequencies.

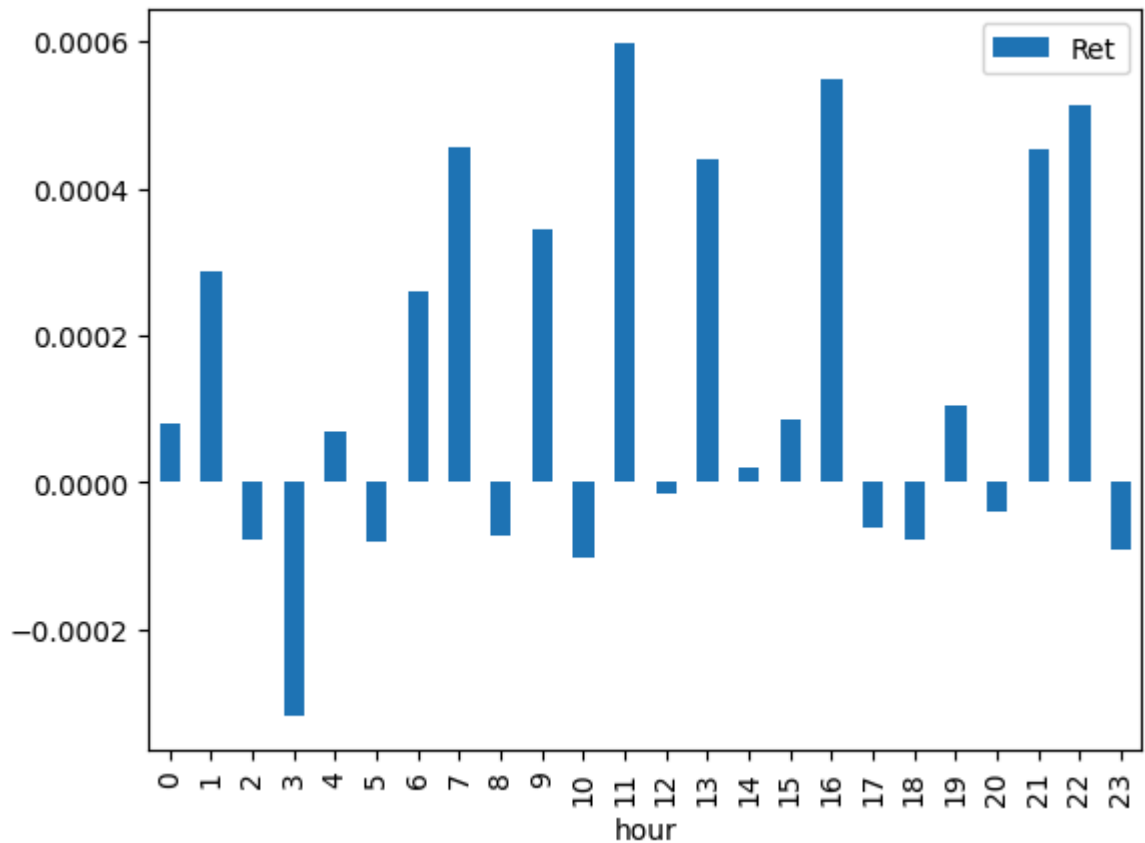
Furthermore, is there any kind of stationarity in the data? It may be the case that BTC prices always climb (or fall) at specific hours or during specific days of the week. This can be especially relevant for prediction tasks that rely on sequences or memory:

```
In [10]: df["year"] = df["date"].dt.year
      df["month"] = df["date"].dt.month
      df["day"] = df["date"].dt.day
      df["week"] = df["date"].dt.isocalendar().week
      df["weekday"] = df["date"].dt.weekday
      df["hour"] = df["date"].dt.hour
```

Let's look at things by hour of the day. We could spend a long time on all these tests, but for the sake of time, we will simply look at average returns during specific hours. Feel free to play with this data by yourselves!

```
In [11]: df_hour = df[["Ret", "hour"]]
      df_hour.groupby(by="hour").mean().plot(kind="bar")
```

```
Out[11]: <Axes: xlabel='hour'>
```

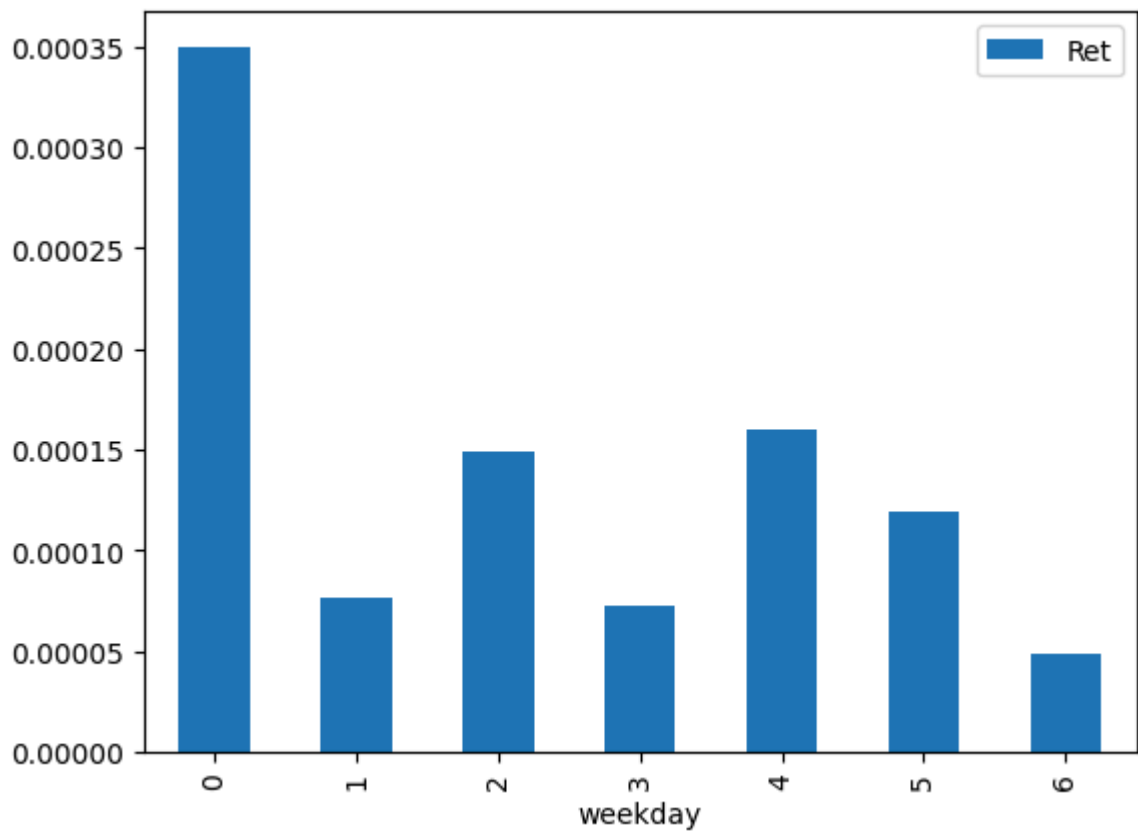


Well, it seems that, on average, BTC prices usually tend to go up at 21h and 22h, while they usually fall by 3am. Note that these are very simple observations and have nothing to do with any kind of causal interpretation you can think of.

Let's see what happens by day of the week:

```
In [12]: df_weekday = df[["Ret", "weekday"]]
df_weekday.groupby(by="weekday").mean().plot(kind="bar")
```

```
Out[12]: <Axes: xlabel='weekday'>
```

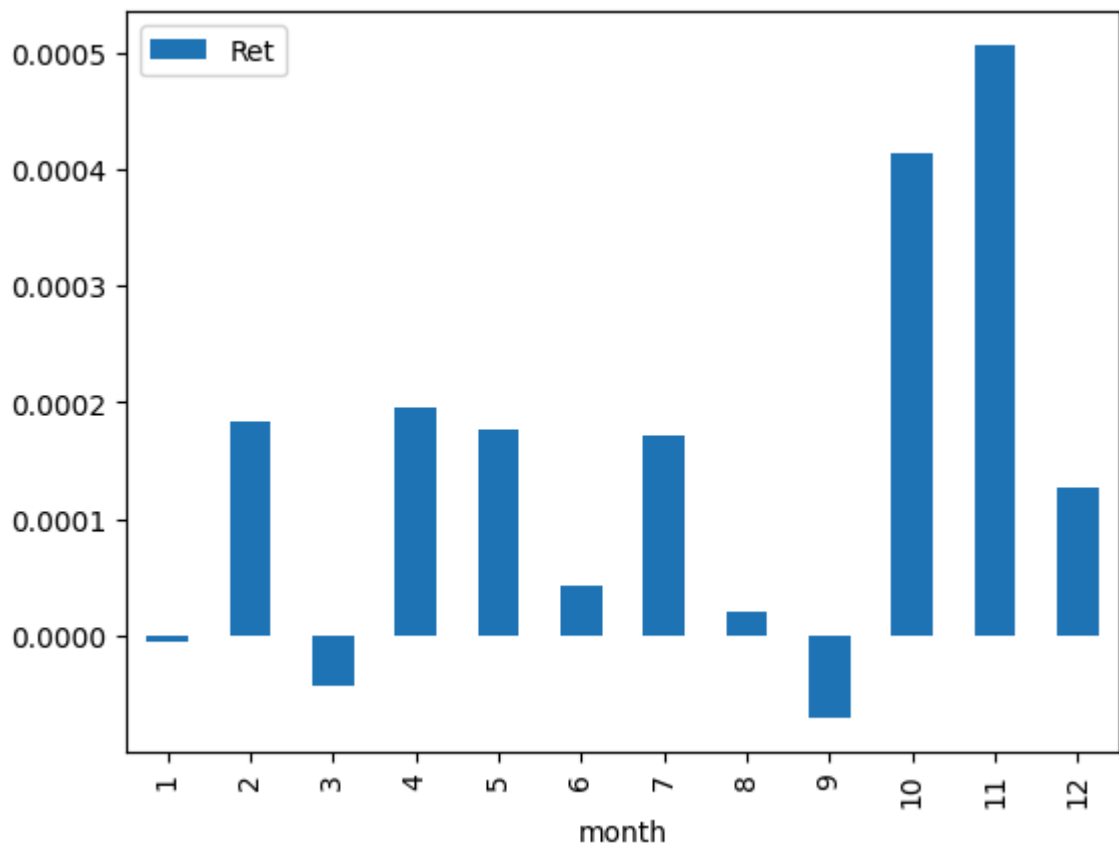


Similar observations can be drawn from this information. However, remember that no causal claim can be made from this!

What about month of the year?

```
In [13]: df_month = df[["Ret", "month"]]  
df_month.groupby(by="month").mean().plot(kind="bar")
```

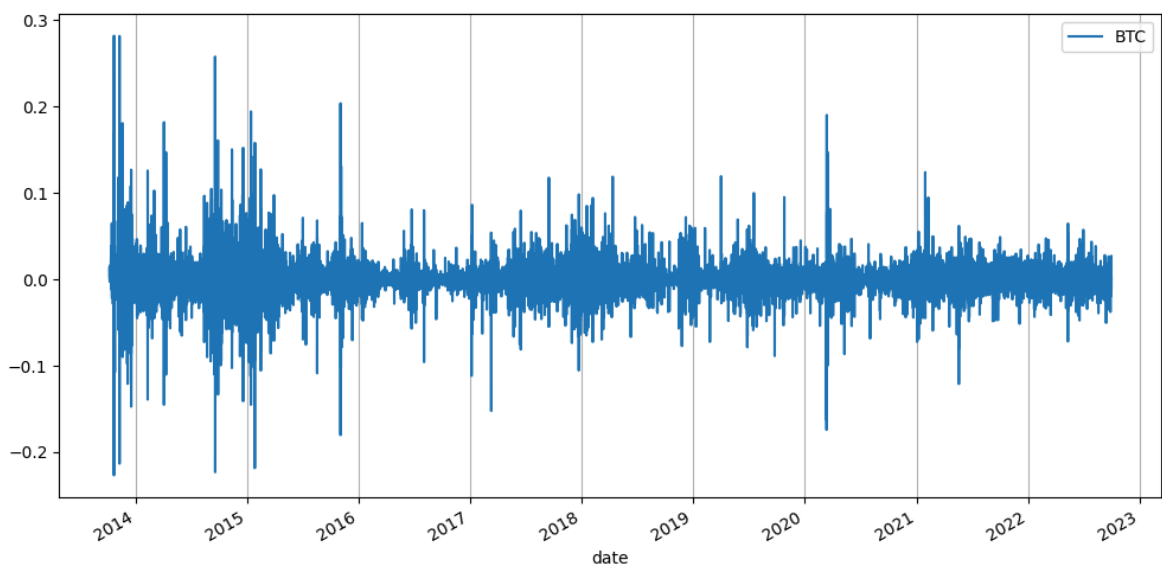
```
Out[13]: <Axes: xlabel='month'>
```



Once again, we can observe that, in the period analyzed, BTC returns were especially positive during October and November. But one more time, no causal inference can be made from this data.

Lastly, let's see how hourly returns evolved through time:

```
In [14]: fig = plt.figure(figsize=(12, 6))
ax = plt.gca()
df.plot(x="date", y="Ret", label="BTC", ax=ax).grid(axis="x")
```



Here, we can see how returns have become less volatile with time. This makes a lot of sense, as the incorporation of major institutional investors to the BTC market propitiated

an increase in market liquidity, leading to lower price impact of trades and fewer price swings. But again, no causal interpretation of the results so far.

So, now that we know a little bit more about how BTC prices fluctuate and the main 'stylized facts' of the data, let's see if a Neural Network based on LSTM can do a good job in predicting future prices once trained.

4. Forecasting

Let's build our LSTM-based prediction model!

The first thing that we are going to do with this large dataset is to keep only the variables we will be using in our model, selecting a chunk of the data to reduce computational time, and replacing missing values:

```
In [15]: df = df[["date", "close", "volume", "trades"]]
df["Ret"] = df["close"].pct_change()
df["year"] = df["date"].dt.year
del df["close"]
df = df.loc[(df["year"] >= 2021)] # To reduce the computational time
```

/tmp/ipykernel_27/800848297.py:2: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df["Ret"] = df["close"].pct_change()
```

/tmp/ipykernel_27/800848297.py:3: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df["year"] = df["date"].dt.year
```

Given the way Kraken data is structured, we may have, especially in the "old days" a bunch of missing values in returns, volume, or trades. This is simply because when the market was not so liquid (or at very specific instances), there may not be a trade happening, and therefore, we won't have information for that variable-hour. For that reason, we are going to replace returns, volume, and trades with zeroes.

Please note that this is not a usual good practice with financial data unless, as in this case, we do have an explanation for the missing data.

```
In [16]: df["Ret"] = df["Ret"].fillna(0)
df["volume"] = df["volume"].fillna(0)
df["trades"] = df["trades"].fillna(0)
```

So, this is finally how our data looks:

```
In [17]: df.head()
```

```
Out[17]:
```

	date	volume	trades	Ret	year
52585	2021-01-01 00:00:00	272.113148	2071	0.003792	2021
52586	2021-01-01 01:00:00	718.037476	3675	0.013203	2021
52587	2021-01-01 02:00:00	274.612863	1433	-0.007229	2021
52588	2021-01-01 03:00:00	86.226981	750	0.003492	2021
52589	2021-01-01 04:00:00	330.509906	1494	-0.002590	2021

We are going to work with these variables and construct a number of different predictors, in a similar fashion to what we have done in previous notebooks of the course.

Specifically, we will compute the 1-hour, 10-hour, 50-hour, and 25-hour cumulative return. The latter (25-hour) is the one we will try to predict using an LSTM model:

```
In [18]: df = df.reindex(
    columns=[
        "date",
        "Ret",
        "volume",
        "trades",
        "year",
        "month",
        "day",
        "week",
        "weekday",
        "hour",
    ]
)
df = df[["date", "Ret"]]

df["Ret_10"] = df["Ret"].rolling(10).apply(lambda x: np.prod(1 + x / 100) - 1)
df["Ret_50"] = df["Ret"].rolling(50).apply(lambda x: np.prod(1 + x / 100) - 1)

df["Ret_25"] = df["Ret"].rolling(25).apply(lambda x: np.prod(1 + x / 100) - 1)
df["Ret25"] = df["Ret_25"].shift(-25)
del df["Ret_25"]
df = df.dropna()
```

This is how our working data will look:

```
In [19]: df.head()
```

Out[19]:

	date	Ret	Ret_10	Ret_50	Ret25
52634	2021-01-03 01:00:00	0.003812	0.000300	0.001236	0.000294
52635	2021-01-03 02:00:00	0.004669	0.000103	0.001244	0.000270
52636	2021-01-03 03:00:00	-0.004577	-0.000065	0.001066	0.000233
52637	2021-01-03 04:00:00	0.019479	0.000120	0.001334	-0.000088
52638	2021-01-03 05:00:00	0.014644	0.000235	0.001445	-0.000118

With this data, we will use the 1-hour, 10-hour, and 50-hour cumulative returns of BTC to predict the 25-hour ahead return. Thus, we define our input (X) and output (y) matrices as:

```
In [20]: Xdf, ydf = df.iloc[:, 1:-1], df.iloc[:, -1]
X = Xdf.astype("float32")
y = ydf.astype("float32")
```

As usual, we will need to define the validation, train, and test samples. Also, the window (sequence length) we will use in our LSTM:

```
In [21]: val_split = 0.2
train_split = 0.625
train_size = int(len(df) * train_split)
val_size = int(train_size * val_split)
test_size = int(len(df) - train_size)

window_size = 30

ts = test_size
split_time = len(df) - ts
test_time = df.iloc[split_time + window_size :, 0:1].values

y_train_set = y[:split_time]
y_test_set = y[split_time:]

X_train_set = X[:split_time]
X_test_set = X[split_time:]

n_features = X_train_set.shape[1]
```

And scale the different variables under the usual MinMaxScaler:

```
In [22]: scaler_input = MinMaxScaler(feature_range=(-1, 1))
scaler_input.fit(X_train_set)
X_train_set_scaled = scaler_input.transform(X_train_set)
X_test_set_scaled = scaler_input.transform(X_test_set)

mean_ret = np.mean(y_train_set)

scaler_output = MinMaxScaler(feature_range=(-1, 1))
y_train_set = y_train_set.values.reshape(len(y_train_set), 1)
y_test_set = y_test_set.values.reshape(len(y_test_set), 1)
```

```
scaler_output.fit(y_train_set)
y_train_set_scaled = scaler_output.transform(y_train_set)
```

Once we have all this, we can finally split the data we will use for training and test:

```
In [23]: training_time = df.iloc[:split_time, 0:1].values

X_train = []
y_train = []

for i in range(window_size, y_train_set_scaled.shape[0]):
    X_train.append(X_train_set_scaled[i - window_size : i, :])
    y_train.append(y_train_set_scaled[i])

X_train, y_train = np.array(X_train), np.array(y_train)

print("Shape of training data", X_train.shape, y_train.shape)

X_test = []
y_test = y_test_set

for i in range(window_size, y_test_set.shape[0]):
    X_test.append(X_test_set_scaled[i - window_size : i, :])

X_test, y_test = np.array(X_test), np.array(y_test)

print("Shape of test data", X_test.shape, y_test.shape)
```

Shape of training data (9490, 30, 3) (9490, 1)

Shape of test data (5682, 30, 3) (5712, 1)

Now it is time to define our LSTM model!

- **The LSTM model:**

Note that we also set a seed for Dropout layers (=1234) to replicate results. Feel free to play around with this!

```
In [24]: SEED = 1234

units_lstm = 50
n_dropout = 0.2
act_fun = "relu"

from keras import Sequential
from keras.callbacks import EarlyStopping

model = Sequential()

model.add(
    LSTM(
        units=units_lstm,
        return_sequences=True,
        activation="tanh",
        input_shape=(X_train.shape[1], n_features),
    )
)
model.add(LSTM(units=units_lstm, return_sequences=True, activation="tanh"))
model.add(Dropout(n_dropout, seed=SEED))
```

```

model.add(LSTM(units=units_lstm, return_sequences=True, activation="tanh"))
model.add(Dropout(n_dropout, seed=SEED))

model.add(LSTM(units=units_lstm, return_sequences=False, activation="tanh"))
model.add(Dropout(n_dropout, seed=SEED))

model.add(Dense(units=20, activation=act_fun))
model.add(Dropout(n_dropout, seed=SEED))

model.add(Dense(units=10, activation=act_fun))
model.add(Dropout(n_dropout, seed=SEED))

model.add(Dense(1))

```

/usr/local/lib/python3.11/site-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(**kwargs)
```

How does this model look like? It is basically a series of LSTM layers (with Dropout after some of them) followed by 3 final dense layers:

In [25]: `model.summary()`

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 30, 50)	10,800
lstm_1 (LSTM)	(None, 30, 50)	20,200
dropout (Dropout)	(None, 30, 50)	0
lstm_2 (LSTM)	(None, 30, 50)	20,200
dropout_1 (Dropout)	(None, 30, 50)	0
lstm_3 (LSTM)	(None, 50)	20,200
dropout_2 (Dropout)	(None, 50)	0
dense (Dense)	(None, 20)	1,020
dropout_3 (Dropout)	(None, 20)	0
dense_1 (Dense)	(None, 10)	210
dropout_4 (Dropout)	(None, 10)	0
dense_2 (Dense)	(None, 1)	11

Total params: 72,641 (283.75 KB)

Trainable params: 72,641 (283.75 KB)

Non-trainable params: 0 (0.00 B)

A quick note on the model parameters:

Where does the 20,200 parameters in lstm_1 come from? Well, in order to define the number of parameters in an LSTM layer, we can use the knowledge that we have already regarding how many parameters (i.e., weights) are trained in each of the equations of the model. Given that there are 4 equations (the usual one plus the extra 3 of the different gates in LSTM layers):

$$\text{Number of parameters} = 4 \times (\text{Hidden Units} + \text{Input size}) \times \text{Hidden Units} + \text{Hidden Units}$$

Applying the previous to lstm_1:

$$4 \times (50 + 50) \times 50 + 50 = 20,200$$

Now, we only need to define the usual final features (learning rate, loss function, early stopping criteria, batch size, ...) and train the model:

```
In [26]: hp_lr = 1e-4
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=hp_lr), loss="mean_absolute
)

es = EarlyStopping(
    monitor="val_loss", mode="min", verbose=0, patience=5, restore_best_weights=
)

# fit the models
model.fit(
    X_train,
    y_train,
    validation_split=val_split,
    epochs=50,
    batch_size=64,
    verbose=2,
    callbacks=[es],
)
```

```

Epoch 1/50
119/119 - 9s - 74ms/step - loss: 0.1445 - val_loss: 0.1014
Epoch 2/50
119/119 - 5s - 39ms/step - loss: 0.1365 - val_loss: 0.1010
Epoch 3/50
119/119 - 5s - 44ms/step - loss: 0.1356 - val_loss: 0.1004
Epoch 4/50
119/119 - 5s - 43ms/step - loss: 0.1340 - val_loss: 0.1011
Epoch 5/50
119/119 - 5s - 43ms/step - loss: 0.1345 - val_loss: 0.1007
Epoch 6/50
119/119 - 5s - 43ms/step - loss: 0.1333 - val_loss: 0.1014
Epoch 7/50
119/119 - 5s - 43ms/step - loss: 0.1331 - val_loss: 0.1029
Epoch 8/50
119/119 - 5s - 43ms/step - loss: 0.1326 - val_loss: 0.1020

```

Out[26]: <keras.src.callbacks.history.History at 0x72546b80e690>

Let's see how this performs out of sample using the standard out-of-sample R-squared measured ($R_{\{OOS\}}$) that we have introduced in previous notebooks.

```

In [27]: prediction = model.predict(X_test)
prediction = scaler_output.inverse_transform(prediction)
prediction = prediction.flatten()

values = np.array(y_test[window_size:])
values = values.flatten()

def R2_campbell(y_true, y_predicted, mean_ret):
    y_predicted = y_predicted.reshape((-1,))
    sse = sum((y_true - y_predicted) ** 2)
    tse = sum((y_true - mean_ret) ** 2)
    r2_score = 1 - (sse / tse)

    return r2_score

R2_Campbell = R2_campbell(values, prediction, mean_ret)
print("Out-of-sample R-squared:", R2_Campbell)

#
df_predictions = pd.DataFrame(
    {"Date": test_time.flatten(), "Pred LSTM": prediction, "values": values}
)
df_predictions.head()

```

178/178 ————— 2s 9ms/step
 Out-of-sample R-squared: -0.0005813956678202281

Out[27]:

	Date	Pred LSTM	values
0	2022-02-05 05:00:00	-0.000011	-0.000017
1	2022-02-05 06:00:00	-0.000011	0.000035
2	2022-02-05 07:00:00	-0.000011	0.000060
3	2022-02-05 08:00:00	-0.000012	0.000046
4	2022-02-05 09:00:00	-0.000012	0.000012

As you can see, the performance out of sample of the model doesn't look so bad. Also, remember that we have used a shortened version of the dataset to reduce computational time. We leave it to you to play with the longer version (careful with computational times!) and try to squeeze the prediction power of the model.

5. Backtest

Finally, let's explore how the predictive performance of the model would have worked in the test period if we traded based on the output of the LSTM model!

We will basically trade long (buy BTC) whenever the 25-day prediction is positive and short (sell BTC) when negative.

```
In [ ]: df_predictions["Positions"] = df_predictions["Pred LSTM"].apply(np.sign)
df_predictions["Strat_ret"] = (
    df_predictions["Positions"].shift(1) * df_predictions["values"]
)
df_predictions["CumRet"] = (
    df_predictions["Strat_ret"].expanding().apply(lambda x: np.prod(1 + x) - 1)
)
df_predictions["bhRet"] = (
    df_predictions["values"].expanding().apply(lambda x: np.prod(1 + x) - 1)
)

Final_Return = np.prod(1 + df_predictions["Strat_ret"]) - 1
Buy_Return = np.prod(1 + df_predictions["values"]) - 1

print("Strat Return =", Final_Return * 100, "%")
print("Buy and Hold Return =", Buy_Return * 100, "%")

ax = plt.gca()
df_predictions.plot(x="Date", y="bhRet", ax=ax)
df_predictions.plot(x="Date", y="CumRet", ax=ax)
plt.show()
```

As we can see, our strategy backtest does a pretty decent job over the buy-and-hold strategy. However, it does not deliver a huge return. As always, the usual disclaimers and caveats about backtesting apply!

- **The role of transaction costs**

A final quick note on transaction costs is necessary here. Trading in the market conveys transaction costs such as brokerage fees. Notice that we have not accounted for these costs before, most of the time because when the number of trades in the testing period is low the impact of transaction costs is minimal. However, in this specific case with hourly data, these costs may be relevant. In the future, we will see how we can incorporate these. For now, it is enough if you acknowledge that the returns obtained here may be misleading as they do not account for transaction costs.

6. Conclusion

In this lesson, we have developed our first LSTM model for predicting the prices of Bitcoin. Although we have found quite decent results in terms of the predictive power of the model, as well as the backtest of the strategy, there is still a lot more to be done when implementing LSTM and other modern Recurrent Neural Networks. In the next lesson, we will take a look at bidirectional networks. See you there!

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

DEEP LEARNING FOR FINANCE

MODULE 5 | LESSON 4

LSTMs FOR PREDICTING BTC PRICES

Reading Time	40 minutes
Prior Knowledge	LSTM, Backtesting, Market timing
Keywords	Bidirectional LSTM, BTC, LSTM

1. Introduction

In this lesson, we will deal with bidirectional neural networks using as a benchmark the same setting for Lesson 2.

The idea here is to check whether, under the current architecture, bidirectional LSTMs bring any advantage over the unidirectional ones studied in lesson 2 of this module. Note that the answer to this question, as usual, cannot be extrapolated to other datasets, other architectures, or other time periods. Hence, we recommend that you play around with the notebook once you understand how to use bidirectional LSTMs to see whether you can enhance the performance of the model.

Bidirectional LSTMs have been recently used to try and predict financial market outcomes such as stock prices. Different from returns themselves, prices may follow a somewhat "logical" sequence (e.g., ascending or descending trend), so using bidirectional networks to try and predict these trends seems like a worthwhile and interesting exercise.

You can find a lot of online resources on bidirectional networks for stock price predictions. Some of them simply used the past series of prices (as we will do here), but others incorporate other types of relevant information. Here, we share some of them, in case you want to take a look:

- Althelaya, Khaled, A., et al. "Evaluation of Bidirectional LSTM for Short-and Long-Term Stock Market Prediction." 9th International Conference on Information and Communication Systems (ICICS), Irbid, Jordan, 2018, pp. 151-156, doi: 10.1109/IACS.2018.8355458. <https://ieeexplore.ieee.org/abstract/document/8355458>
- Liu, Bingchun, et al. "Prediction of SSE Shanghai Enterprises Index Based on Bidirectional LSTM Model of Air Pollutants." *Expert Systems with Applications*, vol.

204, 2022.

<https://www.sciencedirect.com/science/article/abs/pii/S0957417422009113>

- Yang, Mo, and Jing Wang. "Adaptability of Financial Time Series Prediction Based on Bilstm." *Procedia Computer Science*, vol. 199, 2022, pp. 18-25.

<https://www.sciencedirect.com/science/article/pii/S1877050922000035>

So, let's get going!

2. Data

The data that we will use during this lesson is exactly the same as for lesson 2. Hence, to keep things short, we redirect you to the notebook in lesson 2 to see the details of the data-gathering process, as well as some Exploratory Data Analysis (EDA).

```
In [ ]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import tensorflow as tf
from keras.layers import LSTM, Bidirectional, Dense, Dropout
from sklearn.preprocessing import MinMaxScaler
```

```
In [ ]: drivepath = "Kraken_BTCUSD_60_3Q2022.csv"
```

```
In [ ]: df = pd.read_csv(drivepath)

# Given the format of the data, we need to also
df["unix"] = pd.to_numeric(df["unix"])
df["date"] = pd.to_datetime(df["unix"], unit="s")
```

Once we have loaded and formatted our data, we will only work with a few variables:

```
In [ ]: df = df[["date", "close", "volume", "trades"]]
```

```
In [ ]: df.tail()
```

As usual, we calculate the returns for each our to, later on, get the cumulative returns for the different periods and replace the missing values for zeroes given what we know about the data set (**remember** it is not correct to blindly replace missing data for 0; we do it here simply because we do know the reason for that missing data):

```
In [ ]: df["Ret"] = df["close"].pct_change()
df["year"] = df["date"].dt.year
df["Ret"] = df["Ret"].fillna(0)
df["volume"] = df["volume"].fillna(0)
df["trades"] = df["trades"].fillna(0)
df.head()
```

And, finally, for the sake of computational time, we will restrict the sample to post-2021:

```
In [ ]: del df["close"]
df = df.loc[(df["year"] >= 2021)] # To reduce the computational time
```

```
df.head()
```

3. Forecasting

Let's now work on our bidirectional LSTM framework!

As always, let's build our different input/output variables. For these, we will take a very similar approach to what we did in lesson 2, where different cumulative returns for past periods were used to predict the 25-hour ahead return:

```
In [ ]: df = df.reindex(
        columns=[
            "date",
            "Ret",
            "volume",
            "trades",
            "year",
            "month",
            "day",
            "week",
            "weekday",
            "hour",
        ]
    )
df = df[["date", "Ret"]]

df["Ret_10"] = df["Ret"].rolling(10).apply(lambda x: np.prod(1 + x / 100) - 1)
df["Ret_50"] = df["Ret"].rolling(50).apply(lambda x: np.prod(1 + x / 100) - 1)

df["Ret_25"] = df["Ret"].rolling(25).apply(lambda x: np.prod(1 + x / 100) - 1)
df["Ret25"] = df["Ret_25"].shift(-25)
del df["Ret_25"]
df = df.dropna()
```

Next, we need to:

- Define the different sets (validation, train and test), as well as the window size.
- Scale input and outputs using the MinMaxScaler

```
In [ ]: val_split = 0.2
train_split = 0.625
train_size = int(len(df) * train_split)
val_size = int(train_size * val_split)
test_size = int(len(df) - train_size)

window_size = 30

ts = test_size
split_time = len(df) - ts
test_time = df.iloc[split_time + window_size :, 0:1].values

Xdf, ydf = df.iloc[:, 1:-1], df.iloc[:, -1]
X = Xdf.astype("float32")
y = ydf.astype("float32")
```

```

y_train_set = y[:split_time]
y_test_set = y[split_time:]

X_train_set = X[:split_time]
X_test_set = X[split_time:]

n_features = X_train_set.shape[1]

scaler_input = MinMaxScaler(feature_range=(-1, 1))
scaler_input.fit(X_train_set)
X_train_set_scaled = scaler_input.transform(X_train_set)
X_test_set_scaled = scaler_input.transform(X_test_set)

mean_ret = np.mean(y_train_set)

scaler_output = MinMaxScaler(feature_range=(-1, 1))
y_train_set = y_train_set.values.reshape(len(y_train_set), 1)
y_test_set = y_test_set.values.reshape(len(y_test_set), 1)
scaler_output.fit(y_train_set)
y_train_set_scaled = scaler_output.transform(y_train_set)

# Times Series
training_time = df.iloc[:split_time, 0:1].values

X_train = []
y_train = []

for i in range(window_size, y_train_set_scaled.shape[0]):
    X_train.append(X_train_set_scaled[i - window_size : i, :])
    y_train.append(y_train_set_scaled[i])

X_train, y_train = np.array(X_train), np.array(y_train)

X_test = []
y_test = y_test_set

for i in range(window_size, y_test_set.shape[0]):
    X_test.append(X_test_set_scaled[i - window_size : i, :])

X_test, y_test = np.array(X_test), np.array(y_test)

print("Shape of test data", X_test.shape, y_test.shape)

```

\

3.1. Model Architecture:

The next step is defining the architecture of the model. We will, again, follow the exact same architecture as in Lesson 2, with one exception: we will use Bidirectional LSTMs.

The way to set up a bidirectional layer in Keras / TensorFlow is extremely easy, but you should be aware of the different possibilities and operations that the bidirectional layer conveys. Here you can access the Keras and TensorFlow documentation for bidirectional layers:

- Keras: https://keras.io/api/layers/recurrent_layers/bidirectional/

- TensorFlow:
https://www.tensorflow.org/api_docs/python/tf/keras/layers/Bidirectional#call-arguments_1

```
In [ ]: SEED = 1234

units_lstm = 50
n_dropout = 0.2
act_fun = "relu"

from keras import Sequential
from keras.callbacks import EarlyStopping

model = Sequential()

model.add(
    Bidirectional(
        LSTM(
            units=units_lstm,
            return_sequences=True,
            activation="tanh",
            input_shape=(X_train.shape[1], n_features),
        )
    )
)
model.add(
    Bidirectional(LSTM(units=units_lstm, return_sequences=True, activation="tanh"
)
model.add(Dropout(n_dropout, seed=SEED))

model.add(
    Bidirectional(LSTM(units=units_lstm, return_sequences=True, activation="tanh"
)
model.add(Dropout(n_dropout, seed=SEED))

model.add(
    Bidirectional(LSTM(units=units_lstm, return_sequences=True, activation="tanh"
)
model.add(Dropout(n_dropout, seed=SEED))

model.add(
    Bidirectional(LSTM(units=units_lstm, return_sequences=False, activation="tan
)
model.add(Dropout(n_dropout, seed=SEED))

model.add(Dense(units=20, activation=act_fun)) # 40
model.add(Dropout(n_dropout, seed=SEED))

model.add(Dense(units=10, activation=act_fun)) # 10
model.add(Dropout(n_dropout, seed=SEED))

model.add(Dense(1))
```

Now we are ready to train the model!

```
In [ ]: hp_lr = 1e-4
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=hp_lr), loss="mean_absolute
)

es = EarlyStopping(
    monitor="val_loss", mode="min", verbose=0, patience=5, restore_best_weights=
)

# fit the models
model.fit(
    X_train,
    y_train,
    validation_split=val_split,
    epochs=100,
    batch_size=64,
    verbose=2,
    callbacks=[es],
)
```

```
In [ ]: model.summary()
```

One of the main issues related to the bidirectional network is that they have many more parameters, as we can see with a quick check of the model summary (feel free to go back to the unidirectional LSTM case in lesson 2 to see the differences):

A quick note on the model parameters

Where do the 60,400 parameters in `bidirectional_1` come from? Well, it is basically the same computation as in the unidirectional case, but noticing that there are 8 (2×4) equations with parameters now:

$$\begin{equation*} \# \text{ of parameters } = 2 \times 4 \times \left(\left(\text{Hidden Units} + \text{Input size} \right) \times \text{Hidden Units} + \text{Hidden Units} \right) \end{equation*}$$

Applying the previous to `bidirectional_1`:

- Hidden units = 50
- Input size = 100

Hence, number of parameters:

$$\begin{equation*} 2 \times 4 \times \left(\left(50 + 100 \right) \times 50 + 50 \right) = 60,400 \end{equation*}$$

So, what do predictions from the bidirectional LSTM model look like?

```
In [ ]: prediction = model.predict(X_test)
prediction = scaler_output.inverse_transform(prediction)
prediction = prediction.flatten()

values = np.array(y_test[window_size:])
```

```

values = values.flatten()

def R2_campbell(y_true, y_predicted, mean_ret):
    y_predicted = y_predicted.reshape((-1,))
    sse = sum((y_true - y_predicted) ** 2)
    tse = sum((y_true - mean_ret) ** 2)
    r2_score = 1 - (sse / tse)

    return r2_score

R2_Campbell = R2_campbell(values, prediction, mean_ret)
print("Out-of-sample R-squared:", R2_Campbell)

# %%
df_predictions = pd.DataFrame(
    {"Date": test_time.flatten(), "Pred LSTM": prediction, "values": values}
)
df_predictions.head()

```

In this case, it seems that the bidirectional model does a pretty poor job in terms of out-of-sample R-squared. Let's see how it performs in a backtest.

4. Backtest

Finally, we perform a simple backtest of the trading strategy resulting from the predictions delivered by the model. As was the case in Lesson 2, we will basically trade long (buy BTC) whenever the 25-day prediction is positive and short (sell BTC) when negative.

```

In [ ]: df_predictions["Positions"] = df_predictions["Pred LSTM"].apply(np.sign)
df_predictions["Strat_ret"] = (
    df_predictions["Positions"].shift(1) * df_predictions["values"]
)
df_predictions["CumRet"] = (
    df_predictions["Strat_ret"].expanding().apply(lambda x: np.prod(1 + x) - 1)
)
df_predictions["bhRet"] = (
    df_predictions["values"].expanding().apply(lambda x: np.prod(1 + x) - 1)
)

Final_Return = np.prod(1 + df_predictions["Strat_ret"]) - 1
Buy_Return = np.prod(1 + df_predictions["values"]) - 1

print("Strat Return =", Final_Return * 100, "%")
print("Buy and Hold Return =", Buy_Return * 100, "%")

ax = plt.gca()
df_predictions.plot(x="Date", y="bhRet", ax=ax)
df_predictions.plot(x="Date", y="CumRet", ax=ax)
plt.show()

```

As one can easily see from the graph above, the backtest based on the bidirectional LSTM prediction model does a terrible job in the period tested (although still better than

a pure buy-and-hold strategy).

Indeed, the bidirectional model does a worse job than the unidirectional one of lesson 2. This is, however, not usually the case, and bidirectional models tend to perform much better than unidirectional ones. Why is that? Well, for starters, bidirectional models contain much more information, more parameters, and more flexibility. So, why do they perform worse in this case? As always, we have obtained these results under a pre-specified set of hyperparameters for a specific architecture. With models with more parameters, greater care is needed to fine-tune those hyperparameters. It is now your turn to play around with these features to see if you can achieve a better out-of-sample prediction power for the bidirectional model!

5. Conclusion

In this lesson, we have extended the LSTM prediction model framework from lesson 2 to consider bidirectional networks. In this example case, we found little improvement (actually an overall worsening) of model performance in the bidirectional case. However, bidirectional LSTMs usually perform better than the unidirectional models.

This has been all for Module 5. In the next module, we will deal with another powerful feature of neural networks: autoencoders. See you there!

Module 5: Lesson 1

GRUs and LSTM



Outline

- ▶ Gated Recurrent Units (GRUs)
- ▶ Reset and Update gates in GRUs
- ▶ Long Short-Term Memory (LSTM)
- ▶ Input, Forget, and Output gates in LSTM

Gated Recurrent Units (GRUs)

One inherent problem that we encountered in standard RNN models is that long sequences can lead to numerical instability and vanishing and exploding gradients.

In practice, there are some pros and cons of using long sequences for prediction tasks, but what we would ideally like is to have flexibility regarding the length and relevancy of the sequence.

For instance, we may face the following situations:

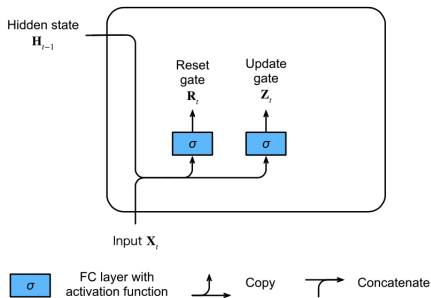
- ▶ An early observation may be highly relevant to explain all future observations → **memory cell?**
- ▶ Some tokens are not relevant → **skip these tokens?**
- ▶ There are logical breaks/parts of sequences (e.g., book chapter) → **reset internal state representation?**

A number of methods have been used to account for these desirable actions. One of the earliest is Long Short-Term Memory (LSTM). We will see this one afterwards and, for now, focus on another simpler one: Gated Recurrent Units (GRUs).

Reset and Update gates in GRUs

The main distinction between vanilla RNNs and GRUs is the fact that the latter incorporate mechanisms to let the network decide when a hidden state should be **updated** or **reset**.

- This is possible thanks to **Reset** and **Update** gates:
 - A reset gate allows us to control how much of the previous state we still want to remember.
 - An update gate allows us to control how much of the new state is a copy of the old one.



Reset and Update gates in GRUs

Mathematically, for a given time step t , minibatch of inputs X_t , and hidden state denoted by H_t , we can mathematically define reset and update gates as:

$$R_t = \sigma(X_t W_{xr} + H_{t-1} W_{hr} + b_r)$$

$$Z_t = \sigma(X_t W_{xz} + H_{t-1} W_{hz} + b_z)$$

Note that activation functions σ are sigmoids, as we want to transform input to values in $(0, 1)$ interval.

The incorporation of these gates into our models will produce a **candidate hidden state** ($\tilde{H}_t$) which moderates the influence that previous hidden states have on new ones:

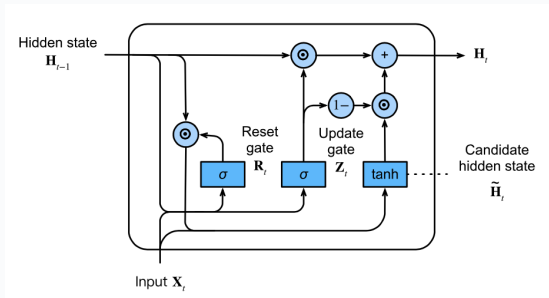
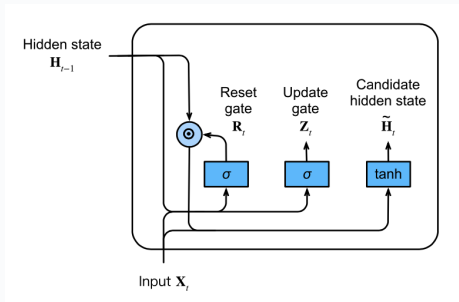
$$\tilde{H}_t = \tanh(X_t W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h)$$

Finally, we can determine our **hidden state** (H_t) by using the candidate one:

$$H_t = Z_t \odot H_{t-1} + (1 - Z_t) \odot \tilde{H}_t$$

Reset and Update gates in GRUs

Here you have a graphical depiction (section 10.1 in the book) of how reset and update gates work:



Note how two interesting things will happen here:

- ▶ When R_t is close to 0 (1), the candidate hidden state is the result of an MLP (vanilla RNN).
- ▶ As Z_t is close to 1 (0), the hidden state approaches the old state (the candidate).

Long Short-Term Memory (LSTM)

In GRUs, reset gates capture short-term dependencies in sequences, while update gates capture long-term ones. LSTM captures these dependencies in a slightly more complex way.

- ▶ An LSTM is ruled by a **Memory cell** (or cell) that stores the additional information and has the same shape of hidden state.
- ▶ The memory cell is governed by a number of gates:
 - **Output gate**: Reads the entries from the cell.
 - **Input gate**: Decides when to read data into the cell.
 - **Forget gate**: Mechanism to reset the content of the cell.

As with GRUs, these gates decide when to remember and when to ignore the inputs in the hidden state. Mathematically:

$$I_t = \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i)$$

$$F_t = \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f)$$

$$O_t = \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o)$$

Input, Forget, and Output gates in LSTM

In implementing these gates in our LSTM, we need to first design our memory cell, which will be governed by these gates that decide how much information is remembered, how much is forgotten, and how much actually passes through to the hidden state.

These decisions depend on the actions of the various gates to arrive at the hidden state:

- First, we define a **candidate memory cell** $\tilde{C}_t$:

$$\tilde{C}_t = \tanh(X_t W_{xc} + H_{t-1} W_{hc} + b_c)$$

- Second, given the results from the forget and input gates, the final memory cell form will be:

$$C_t = F_t \odot C_{t-1} + I_t \odot \tilde{C}_t$$

- Finally, we can compute the hidden state taking into account the output gate:

$$H_t = O_t \odot \tanh(C_t)$$

Input, Forget, and Output gates in LSTM

The following picture illustrates how input, forget, and output gates work:

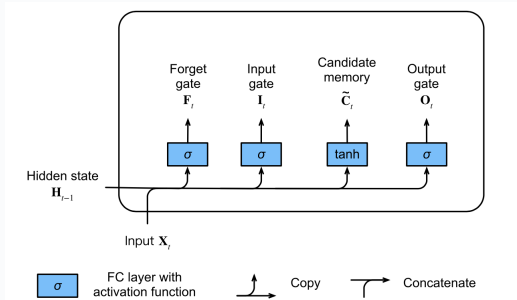


Fig. 10.2.2: Computing the candidate memory cell in an LSTM model.

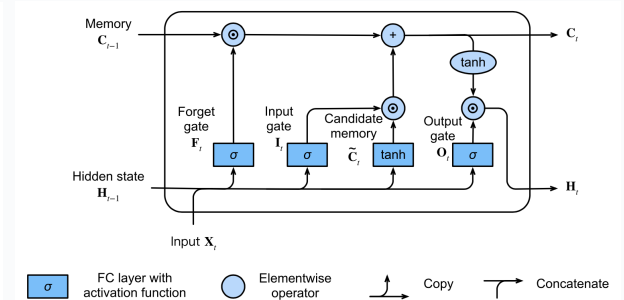


Fig. 10.2.4: Computing the hidden state in an LSTM model.

Summary of Lesson 1

In Lesson 1, we have:

- ▶ Introduced GRUs and LSTM as modern recurrent neural networks
- ▶ Discussed different types of gates to introduce memory (or lack of it) in our models
- ▶ Unveiled the mathematical way to express the functioning of these gates in both GRUs and LSTM.

⇒ **References for this lesson:**

Chapter 10, Sections 10.1 and 10.2 of [Zhang, Ashton, et al. *Dive into Deep Learning*](#).

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will implement all these novel things in practice by investigating the power of LSTM.

Module 5: Lesson 2

LSTMs for predicting BTC prices



Outline

- ▶ The prediction problem at hand
- ▶ EDA: BTC prices and returns
- ▶ LSTM network to predict BTC returns

The prediction problem

As it is usual in finance, our interest one more time lies in predicting short-term future asset returns:

- ▶ Which asset? **Bitcoin (BTC)**
- ▶ Which returns? **25-hour ahead returns**
- ▶ Which inputs? Past cumulative returns for **1-hour, 10-hour, and 50-hour**
- ▶ Which period? **Jan-2021 to Sept-2022**

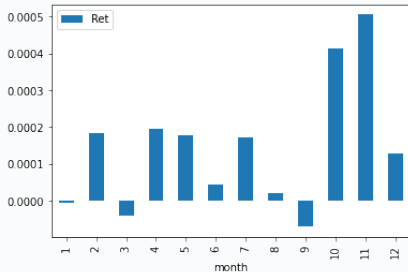
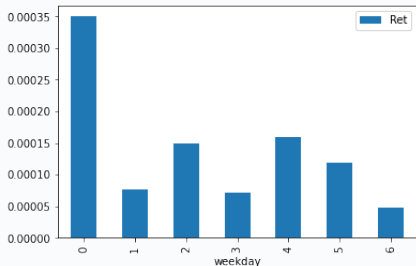
	date	Ret	Ret_10	Ret_50	Ret25
52634	2021-01-03 01:00:00	0.003812	0.000300	0.001236	0.000294
52635	2021-01-03 02:00:00	0.004669	0.000103	0.001244	0.000270
52636	2021-01-03 03:00:00	-0.004577	-0.000065	0.001066	0.000233
52637	2021-01-03 04:00:00	0.019479	0.000120	0.001334	-0.000088
52638	2021-01-03 05:00:00	0.014644	0.000235	0.001445	-0.000118

Exploratory Data Analysis (EDA)

We will be working with what you may call high-frequency data on Bitcoin (BTC) prices. Since we are not so familiar with this data, we may find it useful to perform some simple Exploratory Data Analysis (EDA).

EDA is a particularly helpful way to learn more about the data throughout visualization. This way, we can make more informed choices about how to deal with outliers, the type of scaling needed, etc.

Here are some of the results from EDA:

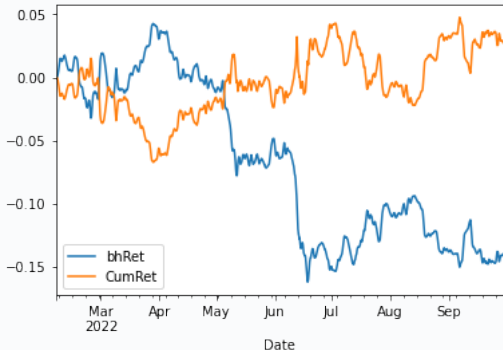


LSTM model to predict BTC returns

In the notebook accompanying this Lesson, we will guide you through the development of an LSTM model. Here you have the summary of the model, as well as the out-of-sample performance of a backtest for the strategy using model predictions:

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 30, 50)	10800
lstm_1 (LSTM)	(None, 30, 50)	20200
dropout (Dropout)	(None, 30, 50)	0
lstm_2 (LSTM)	(None, 30, 50)	20200
dropout_1 (Dropout)	(None, 30, 50)	0
lstm_3 (LSTM)	(None, 50)	20200
dropout_2 (Dropout)	(None, 50)	0
dense (Dense)	(None, 20)	1020
dropout_3 (Dropout)	(None, 20)	0
dense_1 (Dense)	(None, 10)	210
dropout_4 (Dropout)	(None, 10)	0
dense_2 (Dense)	(None, 1)	11

=====
Total params: 72,641
Trainable params: 72,641
Non-trainable params: 0



Summary of Lesson 2

In Lesson 2, we have:

- ▶ Developed our first LSTM Neural Network model
- ▶ Performed Explanatory Data Analysis (EDA) on high-frequency data such as BTC prices
- ▶ Evaluated the predictive power of LSTM and backtest the strategy in test sample.

TO DO NEXT: Go to the accompanying Python Notebook to implement your first LSTM!

In the next lesson, we will learn more about LSTM by introducing bidirectional networks.

Module 5: Lesson 3

Deep and Bidirectional LSTMs

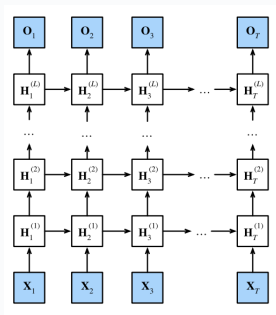


Outline

- ▶ Deep RNNs
- ▶ Bidirectional RNNs and Hidden Markov models
- ▶ Bidirectional LSTMs

Deep RNNs

Although we have practically already worked with RNNs with more than 1 hidden layer, we have not discussed them theoretically. This is indeed quite intuitive to see:



Where each hidden layer l will be expressed as:

$$H_t^{(l)} = \phi_l \left(H_t^{(l-1)} W_{xh}^{(l)} + H_{t-1}^{(l)} W_{hh}^{(l)} + b_h^{(l)} \right)$$

Bidirectional RNNs and Markov models

So far we have assumed that our goal is to model the prediction of the next observation of a sequence given the previous observations. But this can also work both-ways. A clear example arises when dealing with language models:

- ▶ I need to ___
- ▶ I need to ___ something, I'm starving!

What you be more likely to fill in the blank in the first or the second example? Obviously, when we have more information at our disposal, we tend to do better predictions. So does a RNN model!

In Section 10.4.1 of the book you will have an illustrative example of the implications of bidirectionality in Hidden Markov models. If forward (π_{t+1}) and backward (ρ_{t-1}) recursion are given by:

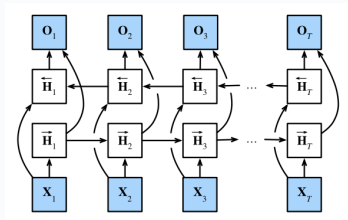
$$\pi_{t+1}(h_{t+1}) = \sum_{h_t} \pi_t(h_t) P(x_t|h_t) P(h_{t+1}|h_t) \qquad \rho_{t-1}(h_{t-1}) = \sum_{h_t} P(h_t|h_{t-1}) P(x_t|h_t) \rho_t(h_t)$$

Then, the conditional probability of a given observation j in the sequence is proportional to:

$$P(x_j|x_{-j}) \propto \sum_{h_j} \pi_j(h_j) \rho_j(h_j) P(x_j|h_j)$$

Bidirectional RNNs

How does bidirectionality work in the context of RNNs?



As you can see, it is relatively simple and intuitive. Similar to the Hidden Markov models, we will need to define the forward ($\vec{H}_t$) and backward ($\overleftarrow{H}_t$) hidden states, which we will do next.

Bidirectional RNNs were first introduced in a paper by Schuster and Paliwal (1997):

Schuster, Mike, and Kuldip K. Paliwal. "Bidirectional recurrent neural networks." *IEEE transactions on Signal Processing* 45.11 (1997): 2673-2681. [Paper here!](#)

Bidirectional RNNs

The forward and backward hidden states in a, say, bidirectional LSTM will be updated as follows:

$$\vec{H}_t = \phi \left(X_t W_{xh}^{(f)} + \vec{H}_{t-1} W_{hh}^{(f)} + b_h^{(f)} \right)$$

$$\overleftarrow{H}_t = \phi \left(X_t W_{xh}^{(b)} + \overleftarrow{H}_{t+1} W_{hh}^{(b)} + b_h^{(b)} \right)$$

This means that we will need to concatenate $\vec{H}_t$ and $\overleftarrow{H}_t$, so that the resulting hidden state, H_t would be of size $n \times 2h$. Once we have H_t , the output layer would be computed as usual:

$$O_t = H_t W_{hq} + b_q$$

where W_{hq} will have size $2h \times q$. Note that we could have different number of hidden units in each of the dimensions, which would alter these dimensions (i.e., it would not be $2h$ anymore)

Adding bidirectionality will largely increase computational costs, as gradients will have long dependency chains. We will explore this in the next lesson, where we will implement a bidirectional LSTM model!

Summary of Lesson 3

In Lesson 3, we have:

- ▶ Introduced formally Deep RNNs
- ▶ Discussed bidirectionality in the context of Hidden Markov models
- ▶ Explored the architecture and main computations of Bidirectional RNNs.

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will put into practice bidirectional LSTM networks by revisiting the BTC example from Lesson 2.